

Lecture 3-3 - Recursion versus iteration

Exercise 1. Implement and verify a method `Mult(x: int, y: int)` that returns the product $m == x * y$ just using the sum

```
method Mult(x: int, y: int) returns (m :int)
  requires 0 <= x && 0 <= y
  ensures m == x * y
{
  var n := 0;
  m := 0;
  while n < y
    invariant 0 <= n <= y
    invariant m == x * n
    {
      m := m + x;
      n := n + 1;
    }
}
```

Exercise 2. Implement and verify a method to compute the best integral approximation to the square root of N by straightforward linear search:

```
method SlowSquareRoot(N: nat) returns (r: nat)
  ensures r * r <= N < (r + 1) * (r + 1)
{
  r := 0;
  while (r + 1) * (r + 1) <= N
    invariant r * r <= N
    {
      r := r + 1;
    }
}
```

Exercise 3. A more efficient way to implement the `SquareRoot()` method is to avoid squaring $r + 1$ each time the loop test is executed by declaring a variable s with an initial value of 1, and adding $s == (r + 1) * (r + 1)$ to the loop's invariant.

Write the new body and prove that the new invariant is preserved by the body by inserting suitable assertions.

Hint: use the fact that:

$$(r + 1) * (r + 1) == (r + 2) * (r + 2) - (2 * r + 3)$$

Solution.

```

method SquareRoot(N: nat) returns (r: nat)
  ensures r * r <= N < (r + 1) * (r + 1)
{
  r := 0;
  var s := 1;
  while s <= N
    invariant r * r <= N
    invariant s == (r + 1) * (r + 1)
    {
      assert s == (r + 1) * (r + 1) <= N;          // by algebra
      assert s == (r + 2) * (r + 2) - 2*r - 3; // by algebra
      assert s + 2*r + 3 == (r + 2) * (r + 2);
      s := s + 2*r + 3;
      assert s == (r + 2) * (r + 2) && (r + 1) * (r + 1) <= N;
      r := r + 1;
      assert s == (r + 1) * (r + 1) && r * r <= N;
    }
  }
}

```

Exercise 4. The method `Div(x, y)` returns the quotient and modulus of the division of `x` by `y`. Implement and verify the algorithm using only subtraction and addition. Then prove that the invariant is preserved by the body by inserting suitable assertions

```

method Div(x: nat, y: nat) returns (q: nat, r: nat)
  requires y > 0
  ensures x == y * q + r && 0 <= r < y
{
  q, r := 0, x;
  while y <= r
    invariant x == y * q + r && 0 <= r
    {
      assert x == y * (q + 1) + r - y;
      r := r - y;
      assert x == y * (q + 1) + r;
      q := q + 1;
      assert x == y * q + r;
    }
  }
}

```

Exercise 5. Add decrease clauses to prove termination of the following methods:

Remark: As the name suggests, when dealing with mutually recursive methods (or functions), it is convenient to designate one as the outer method, which would be exposed in the interface of a package or as a public method of a class, and the other as the inner method, which is an auxiliary function or a private method.

This also suggests how to arrange tuples in the lexicographic order.

```

method RequiredStudyTime(c: nat) returns (hours: nat)

method Outer(a: nat)
  decreases a - 1
{
  if a != 0 {
    var b := RequiredStudyTime(a - 1);
    Inner(a - 1, b);
  }
}

method Inner(a: nat, b: nat)
  decreases a, b
{
  if b == 0 {
    Outer(a);
  } else {
    Inner(a, b - 1);
  }
}

```

Proof:

call	proof obligation for termination

Outer calls Inner	$a - 1 > a - 1, b$
Inner calls Outer	$a, 0 > a - 1$
Inner calls Inner	$a, b > a, b - 1$